Bienvenue au cours “Angular Avancé”

Nom de l’enseignant : Fabrice Meynckens
Promo : 2026-2027
Classe : MPDEV



Copyright© Helios-Consult® Fabrice Meynckens

Objectifs de la session

Services et méthode Inject
- notion de service
- méthode inject
TP : Utilisation de service

Flux et observables en Angular
- Comprendre la programmation asynchrone et réactive dans une application Angular
- Mettre en place un service gérant des flux de données et leur affichage dynamique dans les composants
- RxJS : Observable et Observer
- Async-Pipe
TP : création d'un service simulant des appels asynchrones et actualisant la vue

Lazy loading et composants dynamiques
- Structurer une application modulaire Angular afin d'améliorer son évolutivité
- Mettre en oeuvre l'affichage dynamique de composants pour enrichir l'UX.

Angular Signals
- Angular moderne : des Observables aux données réactives
TP : Le Tracker de Productivité Réactif



Vous trouverez les exemples de code, ce support de cours (dans /public) ainsi que les corrigés des TP sur github :

Cours Angular (https://github.com/swanee-31/Angular-course-tests)

Services et méthode inject

Un composant Angular ne doit gérer que l'interface utilisateur (l'affichage du HTML et l'écoute des clics). Toute la logique métier lourde, le stockage des données partagées ou les appels API doivent être déportés dans des **Services**.

Pour reprendre une analogie classique : le composant est le serveur au restaurant (il prend la commande et apporte le plat), le service est la cuisine (il prépare la donnée).

Qu'est-ce qu'un Service ?

Techniquement, un service n'est rien d'autre qu'une classe TypeScript standard sur laquelle on ajoute le décorateur `@Injectable()`. (@Service avec Angular 22)

Son but principal est d'être **partagé**. Angular utilise le pattern **Singleton** par défaut : lorsqu'un service est fourni à la racine de l'application, Angular n'en crée qu'une seule instance en mémoire. Tous les composants qui demandent ce service recevront exactement la même instance, ce qui permet de partager des données ou un état (comme un panier d'achat ou les informations de l'utilisateur connecté) entre des composants qui ne se connaissent pas.

Dans Angular 22, l'équipe a décidé de s'attaquer au "boilerplate" (le code répétitif). Pendant des années, pour créer un service global (Singleton), il fallait écrire `@Injectable({ providedIn: 'root' })`.

Lien vers la release note
Dependency Injection • Overview • Angular

Le nouveau décorateur @Service() est exactement un raccourci syntaxique (syntax sugar) natif de @angular/core pour ce cas d'usage précis.

- **Ce qui change :** ng generate service crée désormais des classes décorées avec @Service() par défaut. Ce service est automatiquement fourni à la racine (root).
- **Ce qui reste :** @Injectable() n'est pas obsolète. Il reste utilisé sous le capot et reste indispensable si vous avez besoin de configurer des portées d'injection très spécifiques (par exemple, fournir un service uniquement au niveau d'un module paresseux ou d'un composant précis).

La déclaration d'un Service : le standard @Service()
Depuis Angular 22, déclarer un service global est devenu extrêmement lisible grâce au décorateur @Service() importé depuis @angular/core.

Note : Dans d'anciens tutoriels ou sur des projets hérités (avant 2026), vous croiserez systématiquement `@Injectable({ providedIn: 'root' })`.
Les deux approches fonctionnent parfaitement ensemble.

Utiliser la commande “ `ng generate service` “ pour créer un service.

Utilisation du Service et méthode inject

Dans un composant, déclarer l’import de inject et ajouter une propriété (private) de type Service à votre classe.

Vous pouvez ensuite appeler les méthodes du Service depuis le code de votre composant.

src > app > services > TS book.ts > ...

```ts
import { Service } from '@angular/core';

@Service()
export class Book {

    constructor() {}

    hello() {
        console.log('Hello from Book service');
    }
}
```

```ts
import { Component, inject } from '@angular/core';
```

```ts
private bookService: Book = inject(Book);
```

```ts
constructor() {
  console.log('BindingTests component initialized');
  this.bookService.hello();
}
```

Pourquoi utiliser `inject()` ?
- **Disparition des constructeurs :** Le code est beaucoup plus propre et plat.
- **Héritage facilité :** Si vous étendez des classes, vous n'avez plus à vous battre avec la méthode `super()` pour faire passer les dépendances de l'enfant vers le parent.
- **Au-delà des classes :** `inject()` peut être utilisé dans de simples fonctions TypeScript. C'est ce qui a permis de remplacer les anciens *Guards* de routage (qui étaient des classes lourdes) par de simples fonctions modernes.

TP : Utilisation de services

Dans le projet initial (“Ma librairie”), modifier le code afin que le composant affichant les livres récupère la liste des livres depuis un service.

Flux et observables

- Appréhender le changement de paradigme entre la programmation impérative et réactive.
- Comprendre le fonctionnement fondamental de RxJS (Observables, Observers, Subjects).
- Créer un service Angular agissant comme un "mini-store" réactif.
- Consommer ces flux de données proprement et dynamiquement dans l'interface utilisateur.

Programmation Asynchrone vs Reactive

Comprendre la programmation asynchrone et réactive

La programmation réactive consiste à concevoir une application qui **réagit** aux changements d'état plutôt que de les solliciter activement. Au lieu que le composant demande "Quelle est la donnée actuelle ?", le service lui dit "Voici la nouvelle donnée, mets-toi à jour".

En JavaScript classique, l'asynchrone est souvent géré par des **Promesses**. RxJS introduit les **Observables**, qui sont bien plus puissants :

Caractéristique	Promesse (Promise)	Observable (RxJS)
Valeurs renvoyées	Une seule valeur (ou erreur), puis se termine.	De 0 à l'infini (flux continu).
Exécution	Eager (S'exécute dès sa création).	Lazy (Ne s'exécute que si quelqu'un fait un .subscribe()).
Annulable	Non (une fois lancée, on ne peut pas l'arrêter).	Oui (via la méthode .unsubscribe()).
Manipulation	then(), catch(), finally()	Des dizaines d'opérateurs (map, filter, debounceTime...).

Le concept de flux de données (Data Streams)

Il faut passer d'une **logique impérative** (le code demande la donnée de manière séquentielle) à une l**ogique réactive** (le code s'abonne à une source et réagit à chaque nouvelle émission).

Même avec l'arrivée des **Signals** dans les versions récentes d'Angular pour la gestion d'état, RxJS reste le moteur incontournable pour la gestion des événements asynchrones complexes.

L'idée centrale est que **tout peut être représenté sous forme de flux dans le temps**.

- **La métaphore du tapis roulant :** Un flux de données est comme un tapis roulant d'usine. Les données (les boîtes) arrivent les unes après les autres. En bout de chaîne, l'Observer (l'ouvrier) décide quoi faire avec chaque boîte.
- **L'unification de l'asynchrone :** En programmation classique, on gère différemment un clic de souris (`addEventListener`), une minuterie (`setInterval`) et une requête réseau (`fetch`). En RxJS, tout devient un Observable. Le développeur manipule donc des événements locaux ou distants exactement avec la même syntaxe et les mêmes opérateurs.
    - *Un flux de clics :* `---clic---clic---------clic---|>`
    - *Un flux HTTP :* `------(réponse JSON)|>`
    - *Un flux Timer :* `---1---2---3---4---5...>`

L'intégration native de RxJS dans l'écosystème Angular

Angular n'a pas juste ajouté RxJS comme une librairie tierce optionnelle ; le framework a été architecturé autour de lui. Voici les trois piliers où vous croiserez RxJS au quotidien :

- **`HttpClient` :** Toutes les méthodes (`get`, `post`, `put`, etc.) retournent des Observables, et non des Promesses. Même si une requête HTTP ne renvoie qu'une seule réponse, l'utilisation de l'Observable permet de bénéficier des *Retries* (relancer la requête en cas d'échec réseau) et de l'annulation automatique si l'utilisateur change de page avant la fin du chargement.

- **Le `Router` (Routage dynamique) :** Lorsque l'on navigue de `/user/1` vers `/user/2`, le composant Angular n'est pas détruit puis recréé. Seul le paramètre d'URL change. L'objet `ActivatedRoute.params` est un Observable. Les composants s'y abonnent pour être notifiés du changement d'ID en temps réel et recharger les données du nouvel utilisateur sans recharger la vue complète.

- **`ReactiveFormsModule` (Formulaires réactifs) :** Chaque champ de formulaire (`FormControl`) expose des propriétés sous forme de flux.
    - `valueChanges` : Émet un nouvel événement à chaque frappe clavier. Permet de créer facilement des recherches instantanées.
    - `statusChanges` : Émet un flux indiquant si le champ est `VALID`, `INVALID` ou `PENDING`, parfait pour désactiver des boutons de soumission en temps réel.

Les briques fondamentales : Observables et Observers

Pour qu'un flux existe, il faut un émetteur et un écouteur.

- **L'Observable :** C'est le tuyau qui transporte les données.
- **L'Observer :** C'est l'objet qui écoute le bout du tuyau. Il implémente 3 méthodes de callback :
  - `next(valeur)` : Reçoit une nouvelle donnée.
  - `error(erreur)` : Gère une exception (ce qui arrête le flux).
  - `complete()` : Est notifié que le flux est définitivement terminé.

Les Subjects : Créer son propre flux

Un Observable standard est souvent créé à partir d'un événement externe (comme le `HttpClient`). Mais si vous voulez **créer** et **contrôler** votre propre flux de données (ex: l'état d'un panier), vous devez utiliser un **Subject**.

Le plus utilisé est le **`BehaviorSubject`**. Sa particularité :

1. Il nécessite une **valeur initiale**.
2. Il mémorise toujours la **dernière valeur émise**. Tout nouveau composant qui s'y abonne reçoit immédiatement cette dernière valeur.

```
import { Observable, Subscription } from 'rxjs';
```

```
export class ObservableTest implements OnDestroy {
  private monFluxSubscription?: Subscription;

  // On définit le "plan" d'exécution
  monFlux$ = new Observable(subscriber => {
    subscriber.next("Pomme"); // Émet une valeur
    subscriber.next("Banane"); // Émet une autre valeur

    setTimeout(() => {
      subscriber.next("Cerise");
      subscriber.complete(); // Annonce la fin du flux
    }, 2000);
  });
  // À ce stade, RIEN ne s'est exécuté.

  constructor() {
    // On s'abonne au flux pour déclencher son exécution
    this.monFluxSubscription = this.monFlux$.subscribe({
      next: (fruit) => console.log('Reçu :', fruit),
      error: (err) => console.error('Oups :', err),
      complete: () => console.log('Terminé, plus de fruits !')
    });
  }

  ngOnDestroy(): void {
    this.monFluxSubscription?.unsubscribe();
  }
}
```

La Subscription (Le Contrat et les Fuites de Mémoire)

Le concept : Lorsque vous appelez la méthode `.subscribe()`, RxJS démarre l'Observable et vous renvoie un objet appelé `Subscription`. C'est le contrat qui vous lie au flux. Cet objet n'a qu'un seul but fondamental : **pouvoir être annulé**.

Le danger des fuites de mémoire (Memory Leaks) : En Angular, si un composant s'abonne à un flux infini (comme les événements du Router ou un WebSocket) et que l'utilisateur change de page, le composant est détruit (retiré du DOM)... **mais le flux continue de tourner en tâche de fond !** Si l'utilisateur fait 10 allers-retours sur la page, le flux s'exécutera 10 fois en parallèle, faisant crasher le navigateur.

Il faut donc absolument se désabonner (ou utiliser l'AsyncPipe / les Signals qui le font pour nous)

```
ngOnDestroy(): void {
  this.monFluxSubscription?.unsubscribe();
}
```

TP : Le Simulateur de Sonde Thermique

Contexte : Vous développez un tableau de bord industriel. Vous devez vous connecter à une sonde thermique distante qui envoie des relevés de température en continu. La sonde a un comportement spécifique : elle envoie une température toutes les secondes, elle s'arrête d'elle-même après 5 relevés (fin de cycle), mais elle peut aussi "crasher" (erreur) si la température simulée dépasse un certain seuil critique.

Objectifs pédagogiques :

- Créer un **Observable** de toutes pièces (sans utiliser d'opérateurs de création comme `interval` ou `of`).
- Définir l'objet **Observer** avec ses 3 callbacks (`next`, `error`, `complete`).
- Gérer le contrat de **Subscription** pour pouvoir s'abonner et se désabonner (nettoyage / `unsubscribe`).

Travail à réaliser dans un `SensorComponent` :

1. **Création de l'Observable :**
   - Créez une propriété `sensor$` contenant un `new Observable<number>(observer => { ... })`.
   - À l'intérieur, utilisez un `setInterval` (toutes les secondes) qui génère une température aléatoire (entre 0 et 60°C).
   - Utilisez `observer.next()` pour émettre cette température.
   - Si la température dépasse 50°C, utilisez `observer.error()` avec un message "Sonde en surchauffe", et coupez l'intervalle.
   - Si 5 températures valides ont été émises, utilisez `observer.complete()` et coupez l'intervalle.
2. **L'état du composant (Variables classiques) :**
   - Prévoyez des variables pour stocker : la température actuelle, un statut (ex: "En attente", "Connecté", "Erreur", "Terminé"), et l'objet de `Subscription`.
3. **Les actions (Boutons) :**
   - **Bouton "Démarrer" :** Appelle `.subscribe()` sur l'Observable en lui passant un objet **Observer** complet (gérant `next`, `error` et `complete`). Mettez à jour les variables d'état pour l'affichage.
   - **Bouton "Arrêter l'urgence" :** Appelle manuellement `.unsubscribe()` pour couper l'écoute avant la fin du cycle.
4. **La sécurité :**
   - Implémentez l'interface `OnDestroy` pour garantir le `.unsubscribe()` si l'utilisateur change de page pendant que la sonde tourne.

Async-Pipe

L'**Async Pipe** (`| async`) est l'un des outils les plus puissants et indispensables d'Angular lorsqu'on travaille avec la programmation réactive (RxJS).

Dans une architecture Angular moderne (et robuste), c'est la façon **recommandée par défaut** pour consommer de la donnée asynchrone directement dans la vue (le HTML).

Le problème qu'il résout (L'approche Impérative)

Pour bien comprendre son utilité, il faut regarder comment on ferait *sans* lui. Si un composant veut afficher une donnée provenant d'un Observable, on aura le réflexe naturel de faire ceci :

```
// ❌ L'approche manuelle (Déconseillée)
export class ProfileComponent implements OnInit, OnDestroy {
  userData: any;
  private sub!: Subscription;

  constructor(private apiService: ApiService) {}

  ngOnInit() {
    // 1. On s'abonne manuellement
    this.sub = this.apiService.getUser().subscribe(data => {
      this.userData = data; // 2. On stocke la donnée dans une variable locale
    });
  }

  ngOnDestroy() {
    // 3. On DOIT se désabonner pour éviter les fuites de mémoire
    this.sub.unsubscribe();
  }
}
```

Les défauts : C'est verbeux. Il faut gérer des variables locales (`userData`), gérer le cycle de vie (`ngOnInit`, `ngOnDestroy`), et si on oublie le `unsubscribe()`, l'application aura des fuites de mémoire.

La solution : L'approche Déclarative avec le Pipe

Le Pipe `async` fait tout ce travail fastidieux **automatiquement**, directement depuis le template HTML.

- Il fait le `.subscribe()` tout seul au rendu du composant.
- Il extrait la dernière valeur émise et l'affiche.
- **Magie :** Il fait le `.unsubscribe()` tout seul au moment exact où le composant est détruit (changement de page, condition qui passe à faux).

```
// ✅ L'approche avec Async Pipe
import { Component, inject } from '@angular/core';
import { AsyncPipe } from '@angular/common'; // Requis en Standalone
import { ApiService } from './api.service';

@Component({
  selector: 'app-profile',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    <!-- Le pipe async gère toute la mécanique -->
    <p>Bienvenue {{ (user$ | async)?.name }}</p>
  `
})
export class ProfileComponent {
  // On stocke directement l'Observable (convention : suffixe $)
  user$ = inject(ApiService).getUser();

  // Plus de ngOnInit, plus de ngOnDestroy, plus de variable intermédiaire !
}
```

Les Cas d'Usage Avancés (et les pièges à éviter)

L'utilisation basique `{{ user$ | async }}` a une limite : si on utilise le pipe plusieurs fois dans le même HTML sur un Observable "froid" (comme une requête HTTP), Angular va lancer la requête autant de fois qu'il y a de pipes !

La bonne pratique : L'Alias `as` avec le bloc `@if`

Pour éviter les requêtes multiples, on souscrit une seule fois au niveau d'un bloc de condition, et on sauvegarde le résultat dans une variable locale au template via le mot-clé `as`.

```
<!-- On souscrit UNE SEULE fois à user$.
     Si la donnée arrive, on l'appelle "user" pour tout le bloc en dessous. -->
@if (user$ | async; as user) {
  <div class="card">
    <h1>{{ user.name }}</h1>
    <p>Email: {{ user.email }}</p>
    <img [src]="user.avatarUrl" />
  </div>
} @else {
  <!-- Tant que la donnée n'est pas là, on affiche un loader -->
  <p>Chargement du profil...</p>
}
```

Avec une boucle `@for` : Si l'Observable renvoie un tableau de données, l'utilisation est tout aussi élégante :

```
<ul>
  <!-- tasks$ est un Observable<Task[]> -->
  @for (task of tasks$ | async; track task.id) {
    <li>{{ task.title }}</li>
  }
</ul>
```

Async-Pipe : principes à retenir

Pour synthétise, voici pourquoi l'Async Pipe a dominé Angular pendant si longtemps :

1. **Zéro fuite de mémoire :** Le nettoyage est 100 % garanti par le framework lors de la destruction de la vue.

2. **Code TypeScript purifié :** Le fichier `.ts` ne sert plus qu'à déclarer les flux (`streams`). Il ne manipule plus la donnée de manière impérative. C'est l'essence même de l'architecture réactive.

3. **Compatibilité avec `OnPush` (Performance) :** C'est l'atout caché. Si vous passez votre composant en stratégie de détection `ChangeDetectionStrategy.OnPush` (la norme en Angular moderne), l'Async Pipe appelle automatiquement, sous le capot, la méthode `markForCheck()`. Il prévient Angular : *"Hé, une nouvelle donnée vient d'arriver dans le flux, tu peux rafraîchir cette zone précise de l'écran"*.

Note sur Angular 22+ (Le contexte actuel, septembre 2026) : L'Async Pipe reste la méthode standard pour consommer des observables. Cependant, avec l'avènement des **Signals**, la tendance moderne est d'utiliser la fonction `toSignal(monObservable$)` côté TypeScript. Cela transforme le flux RxJS en un Signal. Dans le HTML, on écrit alors simplement `{{ user().name }}` (sans le pipe async). C'est la prochaine étape logique du cours !

TP : Refactoring Réactif avec l'AsyncPipe

Le code précédent fonctionne, mais il est lourd, verbeux, et nous oblige à manipuler le moteur interne d'Angular (`ChangeDetectorRef`) à la main. Vous allez refactoriser ce composant pour le rendre 100 % "Reactive".

Objectifs pédagogiques :

- Remplacer l'abonnement manuel (`.subscribe()`) par le pipe `async` dans le template HTML.
- Utiliser les opérateurs RxJS (`pipe`, `catchError`, `finalize`) pour réagir aux événements du flux.
- Comprendre comment l'AsyncPipe gère automatiquement le cycle de vie (abonnement, désabonnement, et détection de changement).

Travail à réaliser :

1. **Nettoyage du code TypeScript :**
   - Supprimez `ChangeDetectorRef`, `Subscription`, `ngOnDestroy` et tout appel à `.subscribe()` et `.unsubscribe()`.
   - Créez une propriété `sensorData$: Observable<number> | null = null;`.

2. **La méthode Démarrer (`startSensor`) :**
   - Au lieu de vous abonner, **affectez** votre observable de base à `sensorData$`.
   - Utilisez la méthode `.pipe()` sur cet observable pour intercepter les erreurs et la fin du flux :
     - Utilisez `catchError()` pour mettre à jour le statut en cas de surchauffe (puis retournez un flux vide avec `of(null)` pour ne pas faire crasher l'application).
     - Utilisez `finalize()` pour mettre à jour le statut quand le flux se termine (que ce soit un succès naturel ou une annulation manuelle).

3. **La méthode Arrêter (`stopSensor`) :**
   - Pour vous désabonner, il suffit de "détruire" le flux. Passez simplement `sensorData$` à `null` et mettez à jour votre statut.

4. **Le Template HTML :**
   - Utilisez la syntaxe `@if (sensorData$ | async; as temp)` pour afficher la température.
   - Observez le fonctionnement : vous n'avez plus besoin d'appeler `markForCheck()` pour la température, l'AsyncPipe s'en charge !

Angular Signals

L'arrivée des **Signals** a profondément modernisé la façon dont Angular interagit avec les flux de données. Si `AsyncPipe` reste valide, la fonction `toSignal()` (importée de `@angular/core/rxjs-interop`) est aujourd'hui la méthode recommandée pour lire un flux RxJS dans un composant.

Elle permet de convertir un flux asynchrone (Observable) en une donnée synchrone et réactive (Signal) que le template HTML peut lire naturellement.

Voici comment refactoriser l'exemple du compteur temporel avec cette nouvelle approche.

```ts
import { Component } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { interval } from 'rxjs';

@Component({
  imports: [], // plus besoin d'importer AsyncPipe
  selector: 'app-signal-test',
  styleUrl: './signal-test.css',
  templateUrl: './signal-test.html',
})
export class SignalTest {
// 1. On crée le flux RxJS classique
  private interval$ = interval(1000);

  // 2. On le convertit en Signal.
  // On doit fournir une valeur initiale car un Signal ne peut pas être "vide".
  timer = toSignal(this.interval$, { initialValue: 0 });
}
```

```html
<div class="card">
      <h2>Chronomètre (Version Signals)</h2>

      <!-- On appelle simplement le signal comme une fonction -->
      <p>Secondes écoulées : {{ timer() }}</p>

      <hr>

      <!-- Le bloc @if devient beaucoup plus naturel, sans mot-clé 'as' complexe -->
      @if (timer() > 5) {
        <p>Plus de 5 secondes se sont écoulées ! (Actuellement : {{ timer() }})</p>
      }
</div>
```

Avantages de Signal

Le passage de l'Async Pipe à `toSignal` apporte quatre bénéfices majeurs :

1. La syntaxe du template est purifiée Dans le HTML, on n'écrit plus `{{ timer$ | async }}` mais simplement `{{ timer() }}`. Il n'est plus nécessaire d'importer `AsyncPipe` dans le tableau `imports` du composant. Surtout, les conditions complexes redeviennent simples : on peut écrire `@if (timer() > 5)` sans avoir à extraire la valeur avec un alias `as`.

2. La gestion de la valeur initiale Contrairement à un Observable qui peut ne rien émettre pendant plusieurs secondes (comme une requête HTTP en cours), un Signal **possède toujours une valeur immédiate**. C'est pour cela que `toSignal` demande souvent l'option `{ initialValue: 0 }`. Pendant que l'Observable travaille en fond, le Signal renvoie proprement cette valeur par défaut au lieu d'afficher un vide ou de provoquer une erreur.

3. Le nettoyage reste automatique (Zéro fuite) Tout comme le pipe `async`, `toSignal` est extrêmement sécurisé. Il s'abonne à l'Observable lors de sa création et appelle automatiquement `.unsubscribe()` lorsque le composant est détruit. Aucune fuite de mémoire n'est possible.

4. Une détection des changements (Change Detection) chirurgicale C'est le point clé en Angular 22 : avec l'architecture *Zoneless* (sans `zone.js`), Angular ne vérifie plus toute l'application à chaque événement. Lorsqu'un Signal change de valeur, il notifie uniquement la portion exacte du DOM (la balise `<p>`) qui a besoin d'être mise à jour. C'est un gain de performance massif pour les grosses applications.

État dérivé : computed

Le concept : Très souvent dans une application, une donnée B dépend directement d'une donnée A. Si A change, B doit être recalculé. La fonction `computed()` permet de créer un nouveau Signal en lecture seule, dont la valeur est calculée dynamiquement à partir d'un ou plusieurs autres Signals.

C'est l'équivalent moderne et ultra-optimisé d'un *getter* JavaScript, ou des sélecteurs (`createSelector`) dans Redux/NgRx.

Exemple : Le Panier d'achat

```ts
import { Component, computed, signal } from '@angular/core';

@Component({
  imports: [],
  selector: 'app-computed-cart',
  styleUrl: './computed-cart.css',
  templateUrl: './computed-cart.html',
})
export class ComputedCart {
  // 1. Les états de base (Modifiables via .set() ou .update())
  price = signal(1200);
  quantity = signal(1);

  // 2. Les états dérivés (En lecture seule)
  // Angular détecte automatiquement que ce computed dépend de 'price' et 'quantity'
  totalPrice = computed(() => this.price() * this.quantity());

  // Un computed peut même dépendre d'un autre computed !
  isFreeShipping = computed(() => this.totalPrice() > 2000);

  increase() {
    this.quantity.update(q => q + 1);
  }

  decrease() {
    this.quantity.update(q => Math.max(1, q - 1));
  }
}
```

```html
<p>computed-cart works!</p>
<div class="card">
    <p>Produit : Ordinateur Portable</p>
    <p>Prix unitaire : {{ price() }} €</p>

    <button (click)="decrease()">-</button>
    <span> Quantité : {{ quantity() }} </span>
    <button (click)="increase()">+</button>

    <!-- On lit le signal dérivé exactement comme un signal classique -->
    <h3>Total à payer : {{ totalPrice() }} €</h3>

    @if(isFreeShipping()) {
      <p class="badge">Frais de port offerts !</p>
    }
  </div>
```

Utilité de Computed

L’intérêt de `computed` :

1. **Traçabilité automatique :** Vous n'avez pas besoin de déclarer les dépendances manuellement (comme on le ferait dans un `useEffect` en React). Angular observe quels Signals sont lus à l'intérieur de la fonction et crée le lien tout seul.

2. **Mémorisation (Mise en cache) :** La fonction mathématique n'est recalculée **que** si l'un de ses ingrédients (`price` ou `quantity`) a changé. Si un autre événement rafraîchit le composant, Angular renvoie la valeur en cache instantanément. C'est un gain de performance massif.

3. **Évaluation paresseuse (Lazy) :** Le calcul n'est effectué que lorsque le template HTML (ou un autre code) demande à lire `totalPrice()`. Si la valeur n'est affichée nulle part, le calcul ne se lance jamais.

Signal : effect (gérer les effets de bord)

Le concept : Parfois, un changement d'état doit déclencher une action qui sort du système réactif d'Angular.

Par exemple : sauvegarder une donnée dans le `localStorage`, manipuler manuellement un élément du DOM (ex: initialiser une librairie externe comme Chart.js), ou faire un `console.log` pour débugger. C'est ce qu'on appelle un **effet de bord** (Side Effect).

La fonction `effect()` exécute un bloc de code au moins une fois, puis se réexécute automatiquement à chaque fois qu'un des Signals lus à l'intérieur change de valeur.

```html
<button (click)="toggleTheme()">
    Passer en mode {{ isDarkMode() ? 'Clair' : 'Sombre' }}
</button>
```

```typescript
import { Component, effect, signal } from '@angular/core';

@Component({
  imports: [],
  selector: 'app-effect-test',
  standalone: true,
  styleUrl: './effect-test.css',
  templateUrl: './effect-test.html',
})
export class EffectTest {
  // On initialise le signal en lisant le localStorage (ou faux par défaut)
  isDarkMode = signal(localStorage.getItem('theme') === 'dark');

  constructor() {
    // On déclare l'effet dans le constructeur (contexte d'injection)
    effect(() => {
      // 1. Angular voit qu'on lit this.isDarkMode() ici.
      const dark = this.isDarkMode();

      // 2. Ce code s'exécutera à chaque fois que isDarkMode change.
      console.log('Le thème a changé. Nouveau statut sombre :', dark);

      // 3. Effet de bord (Interaction avec une API du navigateur)
      if (dark) {
        localStorage.setItem('theme', 'dark');
        document.body.classList.add('dark-mode');
      } else {
        localStorage.setItem('theme', 'light');
        document.body.classList.remove('dark-mode');
      }
    });
  }

  toggleTheme() {
    this.isDarkMode.update(current => !current);
  }
}
```

Effect : règles d’or

1. **Exécution asynchrone :** Contrairement à un `computed` qui est calculé de manière synchrone et immédiate, un `effect` est planifié par le framework pour s'exécuter un peu plus tard (au prochain cycle de rendu). Il ne bloque jamais l'interface utilisateur.

2. **Pas d'écriture dans d'autres Signals :** C'est une règle de sécurité cruciale. Par défaut, Angular interdit d'utiliser un `.set()` ou `.update()` à l'intérieur d'un `effect()`. Pourquoi ? Pour éviter de créer des boucles infinies (Signal A modifie Signal B, qui modifie Signal A, etc.). Si une donnée dépend d'une autre, il faut utiliser un `computed`. L'effet ne sert qu'à communiquer avec "l'extérieur" d'Angular.

3. **Nettoyage automatique :** Comme pour l'Async Pipe, lorsqu'un composant contenant un `effect()` est détruit, l'effet est désactivé automatiquement.

TP : Le Tracker de Productivité Réactif

Contexte : Vous devez développer un mini-widget permettant à un utilisateur de suivre l'avancement de ses tâches quotidiennes. Le widget doit mémoriser la progression même si on rafraîchit la page, et afficher des messages d'encouragement dynamiques.

Objectifs pédagogiques : Manipuler `signal` (état modifiable), `computed` (état dérivé dépendant d'autres signaux) et `effect` (effet de bord vers une API externe).

Travail à réaliser dans le composant `ProductivityTrackerComponent` :

1. **L'état (Signal) :**
   - Créez une constante `DAILY_GOAL` fixée à 5 tâches.
   - Créez un signal `completedTasks` initialisé avec la valeur trouvée dans le `localStorage` (clé : `'tasks'`), ou `0` par défaut.
2. **Les calculs (Computed) :**
   - Créez un signal dérivé `progressPercentage` qui calcule le pourcentage d'accomplissement (ex: si 2 tâches sur 5 sont complétées, il vaut `40`).
   - Créez un signal dérivé `message` qui renvoie un texte dynamique basé sur le pourcentage :
     - 0% : *"Prêt à démarrer la journée ?"*
     - < 100% : *"Continuez comme ça, vous y êtes presque !"*
     - 100% (ou plus) : *"Félicitations, objectif atteint ! 🚀"*
3. **Les effets de bord (Effect) :**
   - Implémentez un effet qui se déclenche à chaque modification de `completedTasks` pour sauvegarder la nouvelle valeur dans le `localStorage`.
   - Dans ce même effet, si l'objectif est atteint (>= `DAILY_GOAL`), déclenchez une alerte native dans le navigateur (`window.alert`) disant : *"Bravo, vous avez fini votre journée !"*.
4. **La vue (Template HTML) :**
   - Affichez le pourcentage, le message de motivation, et deux boutons : "+1 Tâche complétée" et "Réinitialiser".
   - Le bouton d'ajout doit être désactivé si l'objectif est déjà atteint.